

BÀI TẬP 2

Nhóm 5

Trần Anh Minh-21000242

Lê Thị Hương-21000684

Lê Quốc Khánh-21002466

Bài tập 1:

Thuật toán Topological Sort

Ý tưởng thuật toán

Thuật toán Topological Sort được thiết kế để sắp xếp các đỉnh của một đồ thị có hướng và không chu trình (DAG) sao cho mọi cạnh đi từ đỉnh phía trước đến các đỉnh phía sau. Ý tưởng chính của thuật toán là sử dụng phương pháp tìm kiếm theo chiều sâu (DFS) để duyệt các đỉnh. Bắt đầu từ một đỉnh chưa được duyệt, thuật toán sẽ đi sâu vào các đỉnh kề cho đến khi không còn đỉnh nào để duyệt. Sau khi hoàn thành việc duyệt các đỉnh kề, đỉnh hiện tại sẽ được đưa vào một ngăn xếp. Bằng cách lặp lại quá trình này cho tất cả các đỉnh chưa được duyệt, cuối cùng ta sẽ thu được thứ tự topo bằng cách đảo ngược nội dung của ngăn xếp. Phương pháp này đảm bảo rằng mỗi đỉnh sẽ xuất hiện trước các đỉnh mà nó có cạnh đi đến.

Mã giả (Pseudocode)

Mã giả thuật toán Topological Sort

Input: Đồ thị có hướng không chu trình (DAG) graph

Output: Một thứ tự topo của các đỉnh trong graph

Function TopologicalSort(graph):

 Đánh dấu tất cả các đỉnh là chưa thăm

 stack $\leftarrow$ ngăn xếp rỗng

foreach đỉnh u trong graph **do**

if u là chưa thăm **then**

 DFS(u)

return stack đảo ngược

Function DFS(u):

 Đánh dấu u là đã thăm

foreach đỉnh kề v của u **do**

if v là chưa thăm **then**

 DFS(v)

 Đẩy u vào stack

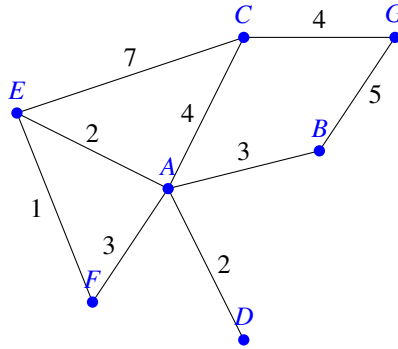
Giải thích

- Khởi tạo tất cả các đỉnh là chưa được thăm và tạo một ngăn xếp rỗng.
- Duyệt qua mỗi đỉnh trong đồ thị:
 - Nếu đỉnh chưa được thăm, thực hiện DFS bắt đầu từ đỉnh đó.
 - Trong quá trình DFS, khi thăm tất cả các đỉnh kề của đỉnh hiện tại, đẩy đỉnh hiện tại vào ngăn xếp.
- Khi đã duyệt toàn bộ đồ thị, thứ tự topo sẽ là kết quả của việc đảo ngược ngăn xếp.

Tất cả các đỉnh và cạnh được duyệt qua một lần nên tổng độ phức tạp về thời gian của thuật toán là $O(|V| + |E|)$.

Bài tập 2:

1. Tìm một cây bao trùm cực tiểu của đồ thị sau sử dụng 2 thuật toán đã học:



Thuật toán Prim

- S : Tập đỉnh đã xét.
- T : Tập các cạnh của cây bao trùm cực tiểu.
- $W(T)$: Tổng trọng số của T .

+) Khởi tạo $S = \{A\}, T = \emptyset, W(T) = 0$.

Tập cạnh có thể chọn: $\{AB, AC, AD, AE, AF\} \Rightarrow$ chọn AD có trọng số nhỏ nhất.

+) $S = \{A, D\}, T = \{AD\}, W(T) = 2$.

Tập cạnh có thể chọn: $\{AB, AC, AE, AF\} \Rightarrow$ chọn AE có trọng số nhỏ nhất.

+) $S = \{A, D, E\}, T = \{AD, AE\}, W(T) = 4$.

Tập cạnh có thể chọn: $\{AB, AC, AF, CE, EF\} \Rightarrow$ chọn EF có trọng số nhỏ nhất.

+) $S = \{A, D, E, F\}, T = \{AD, AE, EF\}, W(T) = 5$.

Tập cạnh có thể chọn: $\{AB, AC, CE\} \Rightarrow$ chọn AB có trọng số nhỏ nhất.

+) $S = \{A, D, E, F, B\}, T = \{AD, AE, EF, AB\}, W(T) = 8$.

Tập cạnh có thể chọn: $\{AC, CE, BG\} \Rightarrow$ chọn AC có trọng số nhỏ nhất.

+) $S = \{A, D, E, F, B, C\}, T = \{AD, AE, EF, AB, AC\}, W(T) = 12$.

Tập cạnh có thể chọn: $\{BG, CG\} \Rightarrow$ chọn CG có trọng số nhỏ nhất.

+) $S = \{A, D, E, F, B, C, G\}, T = \{AD, AE, EF, AB, AC, CG\}, W(T) = 16$.

Tập cạnh có thể chọn: $\emptyset \Rightarrow$ Thuật toán kết thúc ta thu được cây bao trùm cực tiểu $T = \{AD, AE, EF, AB, AC, CG\}$ với tổng trọng số các cạnh là 16.

Thuật toán Kruskal

Tập các cạnh của đồ thị được sắp xếp từ bé đến lớn: $\{EF, AD, AE, AB, AF, AC, CG, BG, CE\}$

Cây bao trùm khởi tạo $T = \emptyset$. Ta duyệt qua các cạnh đã được sắp xếp.

+) Thêm cạnh EF không tạo ra chu trình cho $T \Rightarrow$ Ta cập nhật $T = \{EF\}$.

+) Thêm cạnh AD không tạo ra chu trình cho $T \Rightarrow$ Ta cập nhật $T = \{EF, AD\}$.

+) Thêm cạnh AE không tạo ra chu trình cho $T \Rightarrow$ Ta cập nhật $T = \{EF, AD, AE\}$.

+) Thêm cạnh AB không tạo ra chu trình cho $T \Rightarrow$ Ta cập nhật $T = \{EF, AD, AE, AB\}$.

+) Thêm cạnh AF tạo ra chu trình cho $T \Rightarrow$ Ta không thêm AF vào T .

+) Thêm cạnh AC không tạo ra chu trình cho $T \Rightarrow$ Ta cập nhật $T = \{EF, AD, AE, AB, AC\}$.

+) Thêm cạnh CG không tạo ra chu trình cho $T \Rightarrow$ Ta cập nhật $T = \{EF, AD, AE, AB, AC, CG\}$.

+) Thêm cạnh BG tạo ra chu trình cho $T \Rightarrow$ Ta không thêm BG vào T .

+) Thêm cạnh CE tạo ra chu trình cho $T \Rightarrow$ Ta không thêm CE vào T .

Thuật toán kết thúc ta thu được cây bao trùm cực tiểu $T = \{EF, AD, AE, AB, AC, CG\}$ với tổng trọng số các cạnh là 16.

2. Các khẳng định sau đây đúng hay sai (nếu đúng hãy chứng minh, nếu sai nêu phản ví dụ)

- (a) Mọi cây bao trùm cực tiểu của một đồ thị không bao giờ nhận cạnh có trọng số lớn nhất của đồ thị đó là một cạnh.

Xét T là một cây có trọng số, $|V(T)| = n$ khi đó $|E(T)| = n - 1$.

T liên thông nên ta gọi T' là cây bao trùm cực tiểu của T .

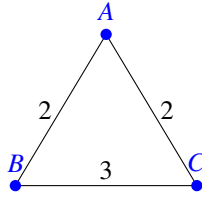
Do $|V(T')| = n$ nên $|E(T')| = n - 1 \implies T \equiv T'$.

Gọi $e \in E(T)$ là cạnh có trọng số lớn nhất của T khi đó $e \in T'$.

Vậy khẳng định sai.

- (b) Nếu G có một chu trình chứa cạnh có trọng số lớn nhất e , thì e không nằm trong bất kì một cây bao trùm cực tiểu nào của G .

Xét đồ thị G sau:



Xét chu trình ABC trong đồ thị G , ta thấy hai cạnh AB và AC có trọng số nhỏ nhất. Do đó, mọi cây bao trùm của đồ thị, bao gồm cả cây bao trùm nhỏ nhất, đều phải chứa ít nhất một trong hai cạnh đó.

Vậy khẳng định sai.

- (c) Thuật toán Prim vẫn cho kết quả đúng với đồ thị có các trọng số âm.

Thuật toán Prim chỉ chú trọng đến việc chọn cạnh có trọng số nhỏ nhất tại mỗi bước, bất kể trọng số đó là dương hay âm, miễn là cạnh trọng số âm không tạo ra chu trình trong cây. Do đó, sự có mặt của cạnh trọng số âm không ảnh hưởng đến nguyên lý hoạt động của Prim.

Vậy khẳng định đúng.

- (d) Cho e là một cạnh có trọng số nhỏ nhất trong đồ thị liên thông G , tồn tại một cây bao trùm cực tiểu của G chứa e .

Ta giả sử mọi cây bao trùm cực tiểu T bất kì của đồ thị G (n đỉnh) đều thỏa mãn $e \notin T$.

Ta thêm cạnh e vào T , khi đó đồ thị mới sẽ có số cạnh lớn hơn $n - 1 \implies$ Đồ thị này tồn tại ít nhất 1 chu trình C . Nếu ta xóa bất kì 1 đỉnh $f \in C$, sẽ thu được đồ thị T' liên thông có $n - 1$ cạnh.

$\implies T'$ là cây bao trùm của G .

Do e có trọng số nhỏ hơn f nên T' có tổng trọng số bé hơn $T \implies T$ không thể là cây bao trùm cực tiểu. Luôn tồn tại một cây bao trùm cực tiểu chứa cạnh có trọng số nhỏ nhất của đồ thị.

Vậy khẳng định đúng.

Bài tập 3:

1. Sử dụng ma trận kề để biểu diễn cấu trúc của đồ thị:

Đồ thị được biểu diễn dưới dạng ma trận kề với kích thước $|V| \times |V|$. Việc tìm cạnh có trọng số nhỏ nhất kết nối đến các đỉnh chưa được thêm vào cây khung tối thiểu đòi hỏi phải duyệt qua toàn bộ ma trận, dẫn đến độ phức tạp $O(|V|^2)$.

$\implies$ Độ phức tạp: $O(|V|^2)$, phù hợp cho đồ thị dày đặc cạnh.

2. Sử dụng danh sách kề để biểu diễn cấu trúc của đồ thị vào Heap Priority Queue để lưu các cạnh có thể được thêm vào cây khung:

Đồ thị được biểu diễn bằng danh sách kề sẽ giảm độ phức tạp về không gian.

Thao tác thêm mỗi cạnh vào Heap Priority Queue có độ phức tạp $O(\log |V|)$, có tất cả $|E|$ cạnh, thời gian thực hiện sẽ là $O(|E| \log |V|)$.

Tương tự, mỗi lần lấy cạnh nhỏ nhất từ Heap Priority Queue cũng có độ phức tạp $O(\log |V|)$, ta có $|V|$ đỉnh cần duyệt, tổng thời gian cho thao tác này là $O(|V| \log |V|)$.

$\implies$ Độ phức tạp: $O(|E| \log |V|)$ (Do $|E| \geq |V| - 1$), phù hợp hơn cho đồ thị thưa cạnh.

3. **Sử dụng danh sách kê để biểu diễn cấu trúc của đồ thị và Fibonacci Heap Priority Queue để lưu các cạnh có thể được thêm vào cây khung:**

Thao tác chèn $|V|$ đỉnh vào đồng tồn $O(1)$ cho mỗi đỉnh, tổng độ phức tạp là $O(|V|)$.

Mỗi lần lấy đỉnh nhỏ nhất từ Fibonacci Heap Priority Queue tốn $O(\log |V|)$, ta cần duyệt $|V|$ đỉnh, tổng chi phí là $O(|V| \log |V|)$.

Đối với mỗi cạnh $|E|$, nếu trọng số cạnh nhỏ hơn trọng số hiện tại của đỉnh kề, thao tác decrease key tốn $O(1)$, tổng cộng là $O(|E|)$.

⇒ Độ phức tạp: $O(|E| + |V| \log |V|)$, phù hợp nhất cho các đồ thị lớn với nhiều đỉnh và cạnh.

Bài tập 4:

Các bước chính để thuật toán Kruskal tìm cây bao trùm cực tiểu

- Sắp xếp các cạnh theo thứ tự tăng dần của trọng số. (Độ phức tạp của bước này $O(|E| \log |E|)$ nếu ta sử dụng thuật toán Mergesort hay Quicksort với $|E|$ là số cạnh của đồ thị)
- Duyệt qua các cạnh, chọn từng cạnh nhỏ nhất mà không tạo chu trình với các cạnh đã chọn trước đó vào cây khung. Mỗi lần kiểm tra chu trình với mỗi cạnh gồm hai thao tác Find và Union, mỗi thao tác có độ phức tạp $O(\alpha(|V|))$. Do đó, bước này có độ phức tạp tổng là $O(|E| \cdot \alpha(|V|))$, gần như $O(|E|)$ trong thực tế.
- Lặp lại quá trình này cho đến khi cây khung có đủ $|V| - 1$ cạnh, với $|V|$ là số đỉnh của đồ thị.

⇒ Độ phức tạp thuật toán là $O(|E| \log |E|)$.

1. **Xét trường hợp trọng số các cạnh là số nguyên dương trong khoảng từ 1 đến $|V|$.**

- Lúc này thay vì sắp xếp các cạnh bằng các thuật toán sắp xếp thông thường ta xây dựng một mảng S có $|V|$ phần tử, mỗi phần tử thứ i của S là danh sách các cạnh có trọng số $i + 1$. Mảng S khi đó là một quan hệ thứ tự của các cạnh theo trọng số. Thao tác hoàn thiện mảng S yêu cầu duyệt qua tất cả các cạnh của đồ thị ⇒ Độ phức tạp $O(|E|)$.
- Hiện tại khi thứ tự trọng số từ bé đến lớn được đảm bảo, ta duyệt từ đầu đến cuối mảng S , mỗi ô duyệt qua từng cạnh để thêm vào cây khung tương tự như Kruskal bình thường. Thuật toán kết thúc khi cây khung đủ cạnh ⇒ Độ phức tạp $O(|V| + |E|) = O(|E|)$ (Do mọi đồ thị liên thông đều có $|E| \geq |V| - 1$).

⇒ Độ phức tạp cả thuật toán được cải thiện thành $O(|E|)$.

2. **Xét trường hợp trọng số các cạnh là số nguyên dương trong khoảng từ 1 đến W .**

Ta xây dựng mảng S với kích thước W và làm tương tự như trên.

⇒ Độ phức tạp cả thuật toán được cải thiện thành $O(|E| + W)$.

Bài tập 5:

1. **Ứng dụng cây bao trùm cực tiểu trong xây dựng mạng lưới điện**

Mục tiêu khi xây dựng mạng lưới điện là đảm bảo cung cấp điện đến tất cả các điểm tiêu thụ với chi phí thấp nhất và duy trì tính liên thông của hệ thống. Hệ thống nên được thiết kế dưới dạng cây và không có chu trình để tránh tình trạng dư thừa kết nối, điều này có thể dẫn đến chi phí cao hơn và khả năng bị tắc nghẽn trong mạng. Áp dụng cây bao trùm cực tiểu giúp xác định các tuyến đường tối ưu cho việc kéo dây điện, đảm bảo mọi khu vực đều được kết nối, đồng thời giảm thiểu vật liệu cần thiết và chi phí bảo trì.

2. **Ứng dụng phân cụm dựa trên cây bao trùm cực tiểu (MST-based Clustering)**

Đây là một kỹ thuật sử dụng cây bao trùm cực tiểu để phân cụm các tập dữ liệu. Để chia thành k cụm, ta tìm cây bao trùm cực tiểu của các điểm dữ liệu và loại bỏ $k - 1$ cạnh có trọng số lớn nhất, tạo ra k thành phần liên thông. Phương pháp này giúp duy trì cấu trúc không gian dữ liệu, đảm bảo rằng các điểm gần nhau được nhóm lại, trong khi những điểm xa hơn sẽ nằm trong các cụm khác nhau. Ta có thể ứng dụng phân cụm dựa trên cây bao trùm cực tiểu trong xử lý ảnh, nhận dạng mẫu, khai phá dữ liệu,...